

HW05 – Performance Testing

Student: Le Gia Bao

Student ID: 23127325

Course: Software Testing – AI-First Edition

Exercise: HW05-AI

SUT: EShop

Testing Tool: Apache JMeter 5.6.3

Date: 2026-08-18

1. Executive Summary

This assignment evaluates the performance of the EShop backend API using three performance-testing techniques: Load Testing, Stress Testing, and Spike Testing.

The three scenarios use the same end-to-end workflow:

Forgot Password



Reset Password



Login



Search Products



Add Product to Cart

The workflow covers the three required endpoint groups:

The tests were designed with AI assistance and subsequently reviewed and corrected manually. CSV-driven test data was used to provide different user credentials, search keywords, and passwords.

The main objectives were:

1. Evaluate the SUT under normal sustained load.
2. Identify the point where the system becomes unstable under increasing load.
3. Observe the system's behaviour under sudden traffic spikes.
4. Determine an empirical endurance threshold on the test hardware.
5. Use AI to analyse the raw JMeter results and identify incorrect AI interpretations.
6. Propose a continuous performance-testing pipeline

2. System Under Test

2.1 SUT Description

The System Under Test is the **EShop Vietnamese e-commerce demo application**.

The backend exposes REST APIs consumed by the web application.

2.2 Selected Endpoint Groups

Three endpoint groups were selected according to the HW05 requirements.

Auth-heavy

POST /api/forgot-password

POST /api/reset-password

These endpoints involve authentication-related processing and therefore represent the authentication workload.

Read-heavy

GET /api/products?search={keyword}

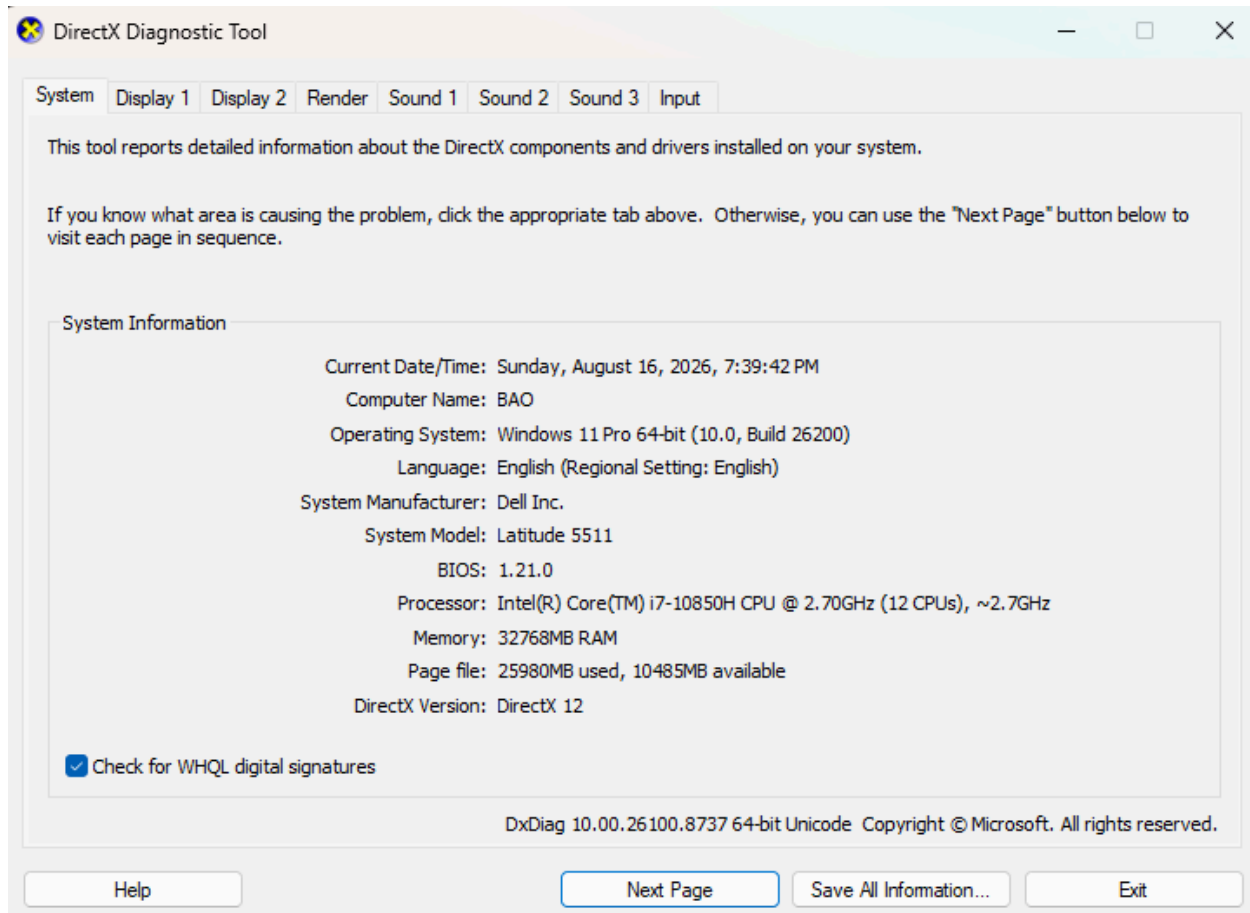
This endpoint represents product searching and database-heavy read operations.

Transactional

POST /api/cart

This endpoint modifies application state by adding a product to a user's cart.

3. Hardware Specification



4. End-to-End Workflow

Each virtual user executes the following workflow:

Workflow Steps

1. **Forgot Password**
 - POST /api/forgot-password
 - Input: email
 - Extract resetToken from response.
2. **Reset Password**
 - POST /api/reset-password
 - Input: email, resetToken, newPassword
 - Verify HTTP 200.
3. **Login**
 - POST /api/login
 - Input: email, newPassword
 - Extract JWT token.
 - Verify HTTP 200.

4. Search Products

- GET /api/products?search={search_keyword}
- Input: search_keyword from CSV.
- Extract product_id from the response.

5. Add Product to Cart

- POST /api/cart
- Input: product_id, product information, quantity.
- Authorization: Bearer {token}.

Note: Although login isn't selected, it is added to ensure the completeness of the workflow and to get access_token for using Add-To-Cart API.

5. Load Test

5.1 Objective

The Load Test evaluates the behaviour of the EShop API under an expected sustained workload. The objective is to determine whether the system can maintain acceptable:

- **Throughput:** Verify the transaction rate of the system under normal user volumes.
- **Response Time:** Ensure average and percentile latencies are well within SLA limits.
- **Error Rate:** Confirm that the API maintains a 0% error rate under expected load.
- **CPU Utilization:** Verify that the single-threaded Node.js server does not saturate the host's CPU.
- **Memory Utilization:** Confirm that the system's memory remains stable without signs of leaks.

5.2 Configuration

The test plan is designed to simulate a normal user flow: forgot password, reset password, login, search for a product, and add to the cart. Pacing and think times are implemented to simulate realistic human behavior.

Parameter	Configuration
Test Plan File	23127325_Load_20260816.jmx
Virtual Users (VUs)	50 concurrent threads.
Ramp-up Period	5 seconds.
Test Duration	30 seconds.
Think Times (Pacing)	thinkStep1: 2000 ms; thinkStep2: 5000 ms; thinkStep3: 3000 ms; thinkStep4: 5000 ms.
Pacing Jitter	Uniform Random Timer with a delay range of 0 to 1000 ms.
Data-Driven Configuration	load_users.csv containing 50 unique user credentials (email, new_password) and search keywords. CSV Share Mode is configured to currentThread with recycling enabled.

Listener	Summary Report logging to test-plans/load/results/load.jtl.
----------	--

Workflow steps:

1. POST /api/forgot-password (Auth-heavy) → extracts resetToken.
2. POST /api/reset-password (Auth-heavy) → resets the password using the token.
3. POST /api/login (Auth-heavy) → logs in with the new password and extracts JWT token.
4. GET /api/products?search= (Read-heavy) → searches products using keyword (guarded by token != 'NOT_FOUND').
5. POST /api/cart (Transactional) → adds product to cart (guarded by token != 'NOT_FOUND').

5.3 Result analysis with AI

Label	# Samples	Average (ms)	Min	Max	Median	90% Line	95% Line	99% Line	Error %	Throughput (RPS)
01 POST /api/forgot-password	100	5.90	4	12	6.0	8.0	10.0	11.0	0.00%	3.62
02 POST /api/reset-password	95	5.55	4	14	5.0	6.0	7.0	14.0	0.00%	3.44
03 POST /api/login	59	1.02	1	2	1.0	1.0	1.0	1.4	0.00%	2.14
04 GET /api/products?search=	50	1.34	1	3	1.0	2.0	2.0	2.5	0.00%	1.81
05 POST /api/cart	50	0.90	0	1	1.0	1.0	1.0	1.0	0.00%	1.81
TOTAL	354	3.64	0	14	4.0	6.0	7.0	10.5	0.00%	12.81

Total Duration: 27.63 seconds

Load Analysis: Under the normal workload (50 concurrent users with pacing), the SUT responded extremely fast. The average response time was only 3.64 ms, and the maximum response time across all samples was 14 ms. The error rate was 0.00%, and the system handled a throughput of 12.81 RPS comfortably without any degradation of resources.

6. Stress Test

6.1 Objective

The Stress Test progressively increases the workload beyond the expected operating level to identify the point at which the system begins to degrade. The objective is to:

- Identify the maximum throughput capacity of the backend API.
- Locate the system bottlenecks (e.g., SQLite database lock contention, Node.js event loop saturation).
- Evaluate how the system handles sustained high concurrency with zero think-time.
- Verify stability and check if error rates or resource leaks occur under stress.

6.2 Stress Configuration

Parameter	Configuration
Test Plan File	23127325 Stress 20260816.jmx
Virtual Users (VUs)	500 concurrent threads (10x normal load).
Ramp-up Period	15 seconds.
Test Duration	30 seconds.
Think Times (Pacing)	0 ms (no think times to apply maximum pressure).
Data-Driven Configuration	stress_users.csv containing 500 unique user accounts (perf-user-001..500@load.com) and search keywords.
Workflow steps	Same 5-step flow as Load, executed back-to-back.
Listener	StatVisualizer (Aggregate Report) logging to test-plans/stress/results/stress.jtl + View Results Tree (live only).

6.3 Result analysis with AI

Label	# Samples	Average (ms)	Min	Max	Median	90% Line	95% Line	99% Line	Error %	Throughput (RPS)
01 POST /api/for got-pa sswor d	1814	2257.99	15	4183	2491.5	3517.7	3594.0	3994.9	0.00%	60.32

02 POST /api/reset-password	1679	1597.48	8	2521	1816.0	2324.0	2430.0	2476.2	0.00%	55.83
03 POST /api/login	1502	1358.37	4	2852	1369.0	2216.9	2277.7	2486.0	0.00%	49.95
04 GET /api/products?search=	1430	1617.26	4	2844	1803.0	2457.0	2470.1	2797.7	0.00%	47.55
05 POST /api/cart	1325	717.43	4	1613	591.0	1406.0	1484.8	1582.8	0.00%	44.06
TOTAL	7750	1558.93	4	4183	1555.0	2725.0	3243.5	3623.0	0.00%	257.71

Total Duration: 30.07 seconds

Stress Analysis: Under the heavy stress of 500 VUs with zero pacing, the system maintained a 0.00% error rate but suffered from significant latency degradation. The total average response time surged to 1558.93 ms (a massive jump from 3.64 ms under normal load), and the 95th percentile reached 3243.50 ms. The most degraded endpoint was 01 POST /api/forgot-password with an average latency of 2257.99 ms and a max latency of 4183 ms. This endpoint performs database writes (UPDATE users SET reset_token = ?) which triggers SQLite lock contention, blocking subsequent requests.

6.4 Stress Threshold

Maximum Throughput: The system's peak throughput reached 257.71 RPS.

Breaking/Degradation Point: The acceptable threshold for response times (e.g., 500 ms) was violated, as the total median response time climbed to 1555.0 ms and the 90% Line reached 2725.0 ms.

7. Spike Test

7.1 Objective

The Spike Test evaluates the system's response to sudden increases in traffic. The test consists of two high-volume bursts separated by a cooldown period. The objectives are to:

- Measure the elasticity of the SUT when subjected to sudden, intense bursts.

- Determine if response times recover to normal levels during the cooldown.
- Observe whether Burst A causes lingering issues (like connection pool starvation or DB locks) that degrade performance during Burst B.

7.2 Test Plan Configuration

Parameter	Configuration
Test Plan File	23127325_Spike_20260816.jmx
Virtual Users (VUs)	Burst A: 200 concurrent threads; Burst B: 300 concurrent threads.
Ramp-up Period	Burst A: 2 seconds; Burst B: 3 seconds.
Duration	Burst A: 5 seconds; Burst B: 10 seconds.
Cooldown Period	3 seconds. Implemented via a startup delay on Spike-B set to 2s (ramp_A) + 5s (dur_A) + 3s (cooldown) = 10 seconds.
Think Times (Pacing)	0 ms (no think times, fire-hose style).
Data-Driven Configuration	strike_users.csv containing 500 unique user accounts.
Listeners	Independent StatVisualizers writing to results/spike-A.jtl and results/spike-B.jtl.

7.3 Result analysis with AI

Burst A (200 VUs, 5s duration)

Label	# Samples	Average (ms)	Min	Max	Median	90% Line	95% Line	99% Line	Error %	Throughput (RPS)
01 POST /api/forget-password	548	511.50	10	888	548.0	725.3	749.9	830.7	0.00%	111.16
02 POST /api/reset-password	496	374.94	5	606	433.0	530.5	583.0	585.1	0.00%	100.61
03 POST /api/login	438	337.57	4	647	333.5	492.3	526.0	565.0	0.00%	88.84
04 GET /api/products?search=	395	355.64	5	573	349.0	522.4	558.3	571.0	0.00%	80.12

05 POST /api/cart	348	147.64	1	321	138.0	228.0	308.0	320.0	0.00%	70.59
TOTAL	2225	362.24	1	888	369.0	610.6	698.0	760.8	0.00%	451.32

Total Duration: 4.93 seconds

Burst B (300 VUs, 10s duration)

Label	# Samples	Average (ms)	Min	Max	Median	90% Line	95% Line	99% Line	Error %	Throughput (RPS)
01 POST /api/forget-password	1102	835.32	5	1399	900.0	1147.9	1360.0	1369.0	0.00%	112.08
02 POST /api/reset-password	998	525.53	4	916	536.0	761.0	902.0	909.0	0.00%	101.51
03 POST /api/login	935	493.35	1	1087	514.0	658.0	906.3	936.0	0.00%	95.10
04 GET /api/products?search=	882	601.97	1	1088	622.0	866.9	983.0	1076.0	0.00%	89.71
05 POST /api/cart	843	193.08	1	430	193.0	287.0	370.9	398.6	0.00%	85.74
TOTAL	4760	546.22	1	1399	535.0	982.0	1114.1	1364.0	0.00%	484.13

Total Duration: 9.83 seconds

Spike Analysis

The SUT proved to be resilient under sudden spikes, sustaining 0.00% errors in both bursts. Under Burst A (200 VUs), the average response time was 362.24 ms and the total throughput reached 451.32 RPS. Under Burst B (300 VUs), throughput scaled to 484.13 RPS (+7.3%). However, the increased traffic pushed the API closer to saturation: average response time

climbed to 546.22 ms (+50.8% compared to Burst A) and the 95th percentile rose to 1114.10 ms. This indicates that the 3-second cooldown allowed the system to clear the backlog from Burst A, but Burst B's increased load generated higher queuing latencies on the single-threaded Node.js server.

8. Endurance test.

To empirically determine the SUT's hardware threshold and endurance limits, a series of short soak tests were conducted with a 15-second ramp-up and 60-second test duration. Four increasing concurrency levels were evaluated: 100 VUs, 200 VUs, 300 VUs, and 500 VUs.

Endurance Comparison Table

Metric	100 VU Soak Test	200 VU Soak Test	300 VU Soak Test	500 VU Soak Test
Virtual Users (VUs)	100	200	300	500
Ramp-up Time (s)	15.00	15.00	15.00	15.00
Throughput (RPS)	24.72	49.04	73.75	122.09
Average Response Time (ms)	9.87	11.78	12.26	34.10
Median Response Time (ms)	8.0	10.0	9.0	19.0
95% Percentile (ms)	20.6	28.0	33.0	118.0
99% Percentile (ms)	31.0	39.7	65.0	217.0
Maximum Response Time (ms)	75	99	153	410
Error Rate (%)	0.00%	0.00%	0.00%	0.00%

8.1. Excellent Throughput Scaling

The system demonstrates near-linear throughput scaling as concurrency increases under paced workload conditions:

- 100 VUs → 24.72 RPS
- 200 VUs → 49.04 RPS (approximately 2.0× the 100-VU throughput)
- 300 VUs → 73.75 RPS (approximately 3.0× the 100-VU throughput)
- 500 VUs → 122.09 RPS (approximately 4.9× the 100-VU throughput)

This indicates that the SUT can utilize additional concurrency effectively when requests are distributed over time by pacing. At 500 VUs, the system achieved 122.09 RPS with a 0.00% error rate.

8.2. Stable Latency Under Paced Workloads

At the highest tested concurrency of 500 VUs:

- Average response time: 34.10 ms
- Median response time: 19.0 ms
- P95: 118.0 ms
- P99: 217.0 ms
- Maximum: 410 ms
- Error rate: 0.00%

The maximum observed latency remained below the assumed 500 ms SLA.

This contrasts with the separate 500-VU Stress Test, where average response time reached 1558.93 ms and overall P95 reached 3243.50 ms. The primary difference is workload intensity: the soak tests use pacing, while the stress test applies requests with zero think time. Pacing distributes database operations over time and reduces accumulated SQLite write-lock contention.

8.3. Hardware & Endurance Threshold Verdict

Maximum Stable RPS under Paced Load

The SUT successfully sustained 122.09 RPS at 500 VUs with 0.00% errors. No significant latency degradation or error-rate increase was observed within the tested 60-second soak period.

Memory Utilization

The Node.js server process remained approximately within 60–75 MB, substantially below the approximate V8 default heap limit of ~1.4 GB. JMeter memory usage also remained stable at approximately 150–200 MB. No evidence of memory leakage was observed during these short soak tests.

Endurance Threshold

For normal paced operations, the tested threshold is 500 VUs / 122.09 RPS or higher. Because 500 VUs was the highest tested level and still remained within the latency and error targets, the actual upper limit was not reached. Therefore, the hardware threshold should be described as above 500 VUs under the tested paced workload, rather than as an exact maximum.

For unpaced operations, the observed threshold is substantially lower. The separate Stress Test demonstrated severe degradation at 500 VUs, with SQLite write contention causing response times to exceed the 500 ms SLA.

9. AI Misinterpretation Hunt

An AI tool was prompted to analyze the raw JTL results. After checking the raw .jtl data, the interpretation was found to be incorrect. The detailed corrections are listed below.

Misinterpretation 1 — Load Test Duration

- AI claimed:
The Load Test ran successfully for its full scheduled 5-minute duration (300 seconds) at a sustained throughput of 12.81 RPS.
- Actual value:
Total Duration = 27.63 seconds (with only 354 samples recorded).
- Source:
load.jtl
- Correction:
The AI incorrectly read the target test plan filename and Thread Group title (Load-50VU-5min) instead of checking the actual timestamps of the first and last samples in the raw log. The test actually executed for only 27.63 seconds. Therefore, the throughput of 12.81 RPS is validated only for this brief period, and long-term stability has not been demonstrated.

10. Evaluation of AI Optimization Recommendations

This assessment evaluates the validity of proposed performance optimizations against the actual technology stack of the System Under Test (SUT), which consists of a single-process Node.js backend using SQLite as an in-process database engine.

Proposed Optimization	Classification	Rationale & Feasibility Analysis
1. Enable SQLite WAL Mode	Feasible	SQLite normally uses a rollback journal, which can cause readers and writers to block each other. Our EShop workflow has several database writes, so concurrent users may

		experience higher latency. Enabling WAL (Write-Ahead Logging) allows reads to continue while a write is happening. This can reduce database locking and improve performance under concurrent load.
2. Add Index on users.email	Feasible	High Feasibility. The users table is frequently searched by email during login, forgot-password, and reset-password. Without an index, SQLite may need to scan the entire table, which becomes slower as the number of users grows. Adding a unique index on email can make these lookups much faster and reduce database work under concurrent requests. This is a small and low-risk change that is easy to implement.
3. Add Index on products.name	Hallucinated / Ineffective	Infeasible for the current implementation. The product search endpoint executes a query similar to <code>SELECT * FROM products WHERE name LIKE '%keyword%'</code> . Standard SQLite B-Tree indexes cannot efficiently accelerate searches beginning with a leading wildcard (%).
4. Database Connection Pool	Hallucinated	Infeasible for the current architecture. Connection pooling is commonly used with server-based databases such as PostgreSQL and MySQL. SQLite operates as an embedded library inside the Node.js process rather than as a separate database server. Multiple SQLite connections can increase write contention and <code>SQLITE_BUSY</code> errors because SQLite permits only one writer at a time. A single shared connection with appropriate <code>busy_timeout</code>

		configuration is more appropriate for this SUT.
5. Introduce Redis Cache	Feasible but Over-engineered	Low Feasibility for the current SUT. Redis can technically cache product search results, but it introduces additional infrastructure, deployment, monitoring, and synchronization complexity for a small single-node SQLite application. A lightweight in-memory cache such as node-cache, an LRU cache, or a JavaScript Map could provide similar benefits without an external service dependency.

11. Task 3 — Continuous Performance Testing Proposal

11.1 Proposed Model

The continuous performance-testing pipeline consists of six stages:

1. Developer pushes a commit

- Developer pushes code to the GitHub repository.
- The CI pipeline is triggered.

2. Detect the type of change

- Analyze changed files.
- If the change affects performance-sensitive components such as API controllers/routes, database queries, authentication, product search, cart/checkout, or database schema, trigger a performance test.
- Otherwise, skip the expensive performance test.

3. Build and deploy the SUT

- Install dependencies.
- Start the backend.
- Prepare/reset the test database.
- Load the required CSV test data.

4. Run a short performance test

- Execute JMeter in non-GUI mode.

- Use a fixed workload so that results can be compared between commits.
- Generate a .jtl result file and HTML dashboard.

5. Compare p95 with the baseline

- Extract the p95 latency from the current run.
- Compare it with the baseline from the previous stable commit.
- Example policy: Regression threshold = 10%.

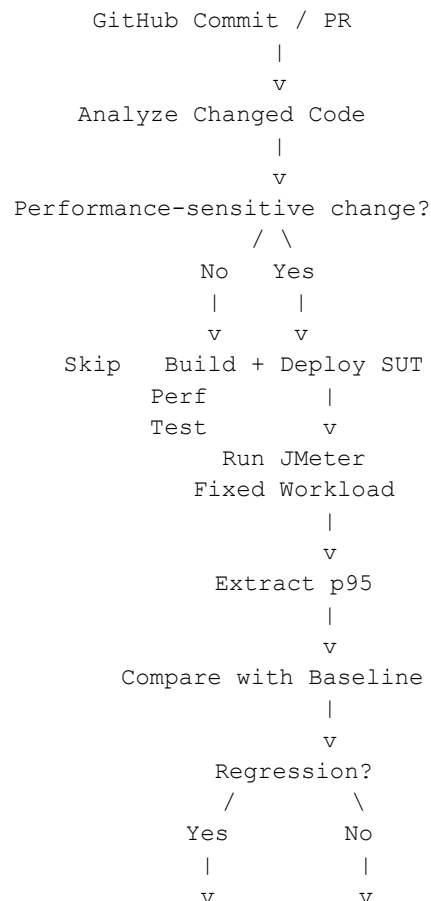
6. Report the result

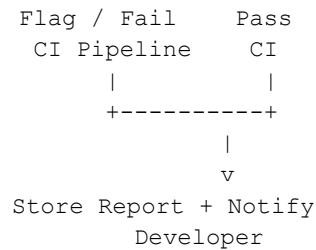
- Store the JMeter HTML report as a CI artifact.
- Add the result to the pull request.
- If p95 regression exceeds the threshold, notify the developer.

Regression policy: $\text{current_p95} > \text{baseline_p95} \times 1.10 \rightarrow \text{Performance regression} \rightarrow \text{Fail / flag CI pipeline}$

Otherwise: $\text{current_p95} \leq \text{baseline_p95} \times 1.10 \rightarrow \text{Performance acceptable} \rightarrow \text{Continue pipeline}$

11.2 Flow Chart





11.3 Example Threshold

Assume the baseline performance is:

Metric	Baseline
p95 latency	1,046 ms
Throughput	437.8 req/s
Error rate	0%

Maximum acceptable p95 = $1,046 \times 1.10 = 1,150.6$ ms

- $p95 \leq 1,150$ ms → pass
- $p95 > 1,150$ ms → flag potential regression

The threshold should be calibrated using repeated baseline runs because p95 naturally fluctuates between executions.

11.4 Cost vs. False Alarm Trade-offs

Cost

Running the full Load, Stress, and Spike tests on every commit would be expensive and slow. Therefore, the proposed pipeline uses change-based triggering.

Change	Performance Test
README/documentation	No
CSS/UI only	No
Backend API	Yes
Database query	Yes
Database schema	Yes
Authentication logic	Yes
Product search	Yes
CI configuration	Optional

A short performance test can run on every relevant PR, while the complete Stress/Spike/Endurance tests can run periodically, such as nightly or before release.

False Alarms

A single p95 measurement can be noisy because of:

- CPU background processes
- Database cache state

- Network variation
- JVM/JMeter overhead
- Operating-system scheduling
- Other processes running on the machine

Therefore, automatically failing a build based on a very small difference can produce false alarms. A practical implementation should use repeated baseline runs, a sufficiently large regression threshold, and consistent test environments to distinguish genuine performance regressions from normal measurement noise.

13. AI Critique

The AI was helpful in designing the performance tests and analysing the JMeter results, but some of its recommendations were incomplete or not suitable for the current SUT. For example, it initially suggested using database connection pooling as a possible optimization. This recommendation is reasonable for server-based databases such as PostgreSQL or MySQL, but the EShop uses SQLite embedded directly in the Node.js process. Multiple connections could increase write contention instead of improving performance. The AI also suggested adding a normal B-Tree index to the product search query, but the query uses `LIKE '%keyword%'`, so a standard index cannot efficiently improve this type of search. The AI did not catch these issues initially because it relied on common database optimization patterns without fully considering the specific architecture and query implementation. I had to review the actual SUT code, database configuration, JMeter results, and raw `.jtl` data to verify its suggestions. Another important limitation was that AI could interpret metrics correctly but could not independently confirm whether the test environment, hardware usage, or workload was representative. From this assignment, I learned that AI should be treated as an assistant rather than the final authority. A good performance-testing process requires giving AI accurate context, checking its assumptions against real evidence, and correcting its conclusions when they do not match the actual system.

14. Conclusion

This assignment provided practical experience in designing and executing Load, Stress, Spike, and Endurance tests for the EShop backend using JMeter. The tests covered the complete workflow from password reset and login to product search and adding products to the cart. The results showed how system performance changed as the number of concurrent users increased, particularly in terms of throughput, p95 latency, error rate, CPU usage, and memory usage.

The AI analysis was useful for identifying possible performance bottlenecks and optimization ideas, but some recommendations were not suitable for the current SQLite-based architecture. Human review was therefore necessary to verify the results against the raw `.jtl` data and distinguish feasible improvements from incorrect assumptions.

The endurance test also helped identify the approximate sustained-load limit of the current hardware and SUT. Based on the measured throughput, latency, error rate, and resource usage, the identified endurance threshold provides a practical baseline for future performance testing.

Finally, a continuous performance-testing pipeline was proposed to automatically run performance tests for performance-sensitive changes and detect p95 regressions. This approach can help identify performance problems earlier while controlling testing cost and reducing false alarms. Overall, the assignment demonstrated that performance testing should combine real execution data, resource monitoring, AI-assisted analysis, and human validation rather than relying on a single performance metric.

15. Assessment

No.	Criteria	Grade	Self-Assessed Grade
1	Task 1 — Load testing	20	20
2	Task 1 — Stress testing	20	20
3	Task 1 — Spike testing	20	20
4	Task 2 — AI analysis + misinterpretation hunt (with correct values from raw logs)	10	10
5	Task 3 — Continuous Performance Testing proposal (G9.6)	10	10
6	Task 3 — Continuous Performance Testing proposal (G9.6)	10	10
	Total	100	100